

PROJECT LOGBOOK

Image Forgery Detection System

Versions V1, V2 and V3 — Evolution & Reflection

Module
Applied Project / Dissertation
Programme
MSc Data Science
Institution
South East Technological University (SETU), Carlow
Students
Manoj Kumar Konda | Ranganayakulu
Academic Year
2025 - 2026
Submission
End of Semester

Table of Contents

1. Product Definition & Overview	3
2. Sustainability & Resilience	4
3. Week-by-Week Logbook	5
Week 1 — V1: MobileNetV2 Baseline	5
Week 2 — V2: EfficientNetB0 Upgrade	6
Week 3 — V3: ManTraNet-Style Architecture	8
4. Data Acquisition & Ethics	10
5. Algorithm Justification	11
6. GitHub Contributions	12
7. Reflection & Evaluation	13

1. Product Definition & Overview

1.1 What We Built

An automated image forgery detection system that identifies digitally manipulated images using deep learning. The system evolved across three versions, each improving accuracy, coverage of forgery types, and explainability of predictions.

1.2 Target Users

- Digital forensics investigators verifying image authenticity
- Journalists and fact-checkers detecting fake news images
- Legal professionals verifying photographic evidence
- Social media platforms detecting manipulated content at scale

1.3 Product Scope

Item	Detail
In Scope	Copy-move, splicing, deepfake detection via web interface
Out of Scope	Video forgery, audio manipulation, real-time streaming
Hardware	Runs on standard CPU — no GPU required
Interface	Browser-based drag-and-drop web application
Output	Verdict + confidence % + pixel-level heatmap (V3)

1.4 Version Summary

Version	Model	Dataset	Forgery Types	Accuracy	New Feature
V1	MobileNetV2 + RBF-SVM	MICC-F220 (220 images)	Copy-Move	~77%	Baseline web app
V2	EfficientNetB0 + LinearSVC	CASIA v2 + CoMoFoD + Columbia (24,459)	Copy-Move + Splicing	81.34%	Batch loading, StandardScaler
V3	ManTraNet-style BayarConv+SRM+U Net	All above + FaceForensics++ (1,751 frames)	Copy-Move + Splicing + Deepfake	In progress	Pixel heatmap + bounding boxes

2. Sustainability & Resilience

Sustainability refers to the system's ability to remain useful, maintainable, and extendable over time. **Resilience** refers to its ability to handle unexpected inputs, failures, and edge cases gracefully.

2.1 Sustainability Objectives

Modular Architecture

Each version of the system is built as a self-contained module. The training pipeline (`train.py`), inference backend (`app.py`), and frontend (`index.html`) are completely separate. This means any component can be upgraded or replaced without affecting the others. For example, upgrading from EfficientNetB0 to ManTraNet in V3 required no changes to the frontend.

Versioned Development

The project follows a deliberate three-version evolution strategy, each version solving a specific limitation of the previous one. This iterative approach ensures the system improves incrementally rather than requiring complete rewrites. GitHub branches (`v1-mobilenetv2`, `v2-efficientnet`, `v3-mantranet`) preserve the full history of every decision made.

Dataset Extensibility

The training pipeline uses a simple folder structure (`Dataset/Original` and `Dataset/Tamper`) that accepts any new dataset without code changes. Adding FaceForensics++ deepfake frames in V3 required only copying files into the Tamper folder — no modification to `train.py` was needed.

Memory Efficient Design

V2 introduced batch-based feature extraction that processes 16 images at a time instead of loading all 24,459 images into RAM simultaneously. This reduced peak memory usage from 13.7GB to approximately 97MB, making the system deployable on standard hardware without expensive cloud infrastructure.

Open Source Dependencies

All libraries used (PyTorch, TensorFlow, Scikit-learn, Flask, OpenCV) are open source with active maintenance communities. There is no dependency on paid APIs or proprietary software, ensuring the system can be run and maintained at zero cost indefinitely.

2.2 Resilience Objectives

Graceful Error Handling

The Flask backend handles all common failure modes gracefully. If no image is uploaded, the API returns a clear error message. If the image cannot be decoded, a descriptive error is returned. If the model fails during inference, the exception is caught and returned as a JSON error rather than crashing the server.

Class Imbalance Handling

V2 and V3 both implement class weight balancing using `compute_class_weight` from scikit-learn. When one class has more images than the other, the model would otherwise become biased toward the majority class. Class weights compensate for this automatically, making the model equally sensitive to both Original and Tampered images regardless of dataset proportions.

Unreadable Image Handling

During batch feature extraction, if any image file is corrupted or unreadable, the system replaces it with a zero-filled array rather than crashing. This ensures training completes even if a small number of dataset files are damaged.

Model Availability Check

The V3 Flask application checks whether `ManTraNet.pt` exists before attempting to load it. If the weights file is missing, the server still starts and returns a helpful error message through the `/model_status` endpoint, allowing the frontend to display a meaningful status to the user rather than showing a 500 error.

No Single Point of Failure

The system has a clear separation between training and inference. If the training process fails partway through, the last best model checkpoint is already saved to disk. The application can still run using the previously saved model. Training and serving are completely independent processes.

Resilience Challenge	How We Addressed It
13.7GB RAM crash	Batch loading — 16 images at a time
RBF-SVM too slow on 24K images	Switched to LinearSVC — seconds not hours
Class imbalance in dataset	<code>compute_class_weight</code> balances training automatically
Corrupted image files	Zero-fill fallback — training continues
Missing model weights	Server starts, <code>/model_status</code> returns clear error
ManTraNet weights unavailable	Implemented architecture from scratch, trained on own data
JPEG compression destroying artifacts	SRM + BayarConv operate on noise residuals, more robust

3. Week-by-Week Logbook

WEEK 1 — Version 1: MobileNetV2 + MICC-F220 Baseline

3.1 Objectives

- Understand the image forgery detection problem domain
- Select an appropriate baseline model and dataset
- Build and deploy a working end-to-end web application

3.2 What We Did

Dataset — MICC-F220

Downloaded the MICC-F220 dataset containing 220 images — 110 original and 110 copy-move forged. This is the standard benchmark dataset for copy-move forgery detection research, originally published by Amerini et al. (2011).

Model — MobileNetV2 + RBF-SVM

Chose MobileNetV2 as the feature extraction backbone due to its lightweight architecture designed for deployment on resource-constrained devices. Used transfer learning — froze all MobileNetV2 weights and used the 1280-dimensional output as image features. Trained an RBF-SVM classifier on these features to distinguish Original from Tampered images.

Web Application

Built a Flask backend with a REST API endpoint /predict that accepts image uploads and returns a JSON response with label and confidence score. Built a drag-and-drop HTML/CSS/JavaScript frontend that displays the result with animated confidence bars.

3.3 Results

Metric	Value
Dataset size	220 images (110 Original, 110 Tampered)
Feature vector	1280 dimensions (MobileNetV2 output)
Test accuracy	~77%
Forgery types detected	Copy-Move only
Training time	< 5 minutes

3.4 Problems Faced

- Small dataset (220 images) limited model generalisation to unseen images
- Model only detects copy-move forgery — cannot detect splicing or deepfakes
- Random internet images almost always predicted as Authentic regardless of content

3.5 Decisions Made

- Expand to multiple larger datasets covering different forgery types
- Upgrade from MobileNetV2 to a more powerful feature extractor
- Add StandardScaler to normalise features before SVM training

WEEK 2 — Version 2: EfficientNetB0 + Multi-Dataset

3.6 Objectives

- Expand dataset to cover more forgery types and more images
- Upgrade model backbone to improve feature quality
- Fix RAM crash caused by loading all images simultaneously
- Improve training speed for large dataset

3.7 Dataset Expansion

Combined three publicly available forgery detection datasets:

Dataset	Source	Images	Forgery Type
CASIA v2	Kaggle	~10,000	Copy-Move + Splicing
CoMoFoD	University of Zagreb	~10,000	Copy-Move (rotation, scaling, flipping)
Columbia Uncompressed	Columbia University DVMM Lab	~4,000	Image Splicing
TOTAL		24,459	Copy-Move + Splicing

3.8 Key Technical Improvements

EfficientNetB0 — Compound Scaling

Replaced MobileNetV2 with EfficientNetB0 which uses compound scaling to simultaneously increase network depth, width, and resolution. This produces 5.1% better ImageNet accuracy with only 1.9M more parameters, generating richer 1280-dimensional feature vectors better suited for detecting subtle forgery artifacts across diverse datasets.

Batch Loading — RAM Crash Fix

The original approach loaded all 24,459 images into RAM at once, requiring 13.7GB and crashing the process. Replaced with batch loading: process 16 images at a time, extract features, free RAM, process next batch. Peak memory usage reduced from 13.7GB to approximately 97MB.

LinearSVC — Speed Improvement

RBF-SVM training on 19,567 samples took 20+ minutes. Replaced with LinearSVC wrapped in CalibratedClassifierCV (for probability support). LinearSVC training completed in under 60 seconds with comparable accuracy, making the full training pipeline practical on a standard laptop.

StandardScaler

Added StandardScaler to normalise all 1280 feature dimensions to zero mean and unit variance before SVM training. Without scaling, features with large absolute values dominate the SVM decision boundary regardless of their actual information content. Scaling improved classification accuracy by approximately 3-4%.

3.9 Results

Metric	V1	V2	Improvement
Dataset size	220	24,459	+111x
Backbone	MobileNetV2	EfficientNetB0	Compound scaling
Image resolution	96 x 96	224 x 224	Better detail
Test accuracy	~77%	81.34%	+4.34%
Forgery types	1 (Copy-Move)	2 (Copy-Move + Splicing)	+1 type
Training time	< 5 min	~25 min (feature extraction)	Acceptable

3.10 Problems Faced

- 13.7GB RAM crash when loading full dataset — fixed with batch loading
- RBF-SVM took 20+ minutes on large dataset — fixed with LinearSVC
- matplotlib plt.show() blocked script execution — fixed with plt.close()
- Flask could not find index.html — fixed with os.path.abspath(__file__)

WEEK 3 — Version 3: ManTraNet-Style with Pixel Heatmap

3.11 Objectives

- Add deepfake detection capability to the system
- Implement pixel-level forgery localisation (show WHERE forgery is)
- Move from classification-only output to segmentation heatmap output

3.12 FaceForensics++ Dataset

Submitted an official data request form to the Technical University of Munich for FaceForensics++. Received approval within 24 hours. Downloaded 50 compressed deepfake videos (~500MB) using the `--sample_only` flag. Extracted 1,751 frames using OpenCV at `FRAME_SKIP=15` (one frame per 15 frames to avoid near-duplicate frames).

3.13 Architecture — ManTraNet-Style

Why ManTraNet-Style?

The official ManTraNet pre-trained weights were no longer publicly available from the original GitHub repository. TruFor weights required Docker with institutional access. Rather than depending on unavailable external weights, we implemented the ManTraNet architectural principles from scratch and trained them on our own combined dataset.

BayarConv2d — Manipulation Artifact Detection

A constrained convolutional layer where the centre pixel weight is forced to equal the negative sum of all surrounding weights. This constraint means the filter can only detect differences between pixels and their neighbours — exactly the pattern left by image manipulation tools that alter local pixel relationships.

SRMConv2d — Noise Residual Extraction

Three fixed mathematical filters borrowed from steganalysis research (Steganalysis Rich Model). These filters extract the high-frequency noise residual of an image — the invisible pattern left by the camera sensor and compression algorithm. When a region is forged, its noise residual pattern is inconsistent with the surrounding authentic region, revealing the boundary of manipulation.

U-Net Decoder — Pixel-Level Heatmap

After the shared encoder processes both noise signals and visual features, a U-Net decoder reconstructs a pixel-level segmentation map. Skip connections from each encoder layer pass to the corresponding decoder layer, preserving spatial precision. The final output is a heatmap the same size as the input image where each pixel receives a forgery probability score. This replaced the LSTM used in the original ManTraNet paper because our task is spatial (2D image localisation) rather than sequential, and U-Net skip connections preserve exact XY coordinates of the forgery boundary.

Dual Output

The model produces two outputs simultaneously: a classification head (Original/Tampered with confidence %) and a segmentation head (pixel heatmap showing WHERE). The combined training loss is classification loss plus 0.5 times segmentation loss.

3.14 Results

Metric	Value
Total dataset	~26,210 images (24,459 + 1,751 deepfake frames)
Forgery types covered	Copy-Move, Splicing, Deepfake
New output	Pixel heatmap + bounding boxes around forged regions
Architecture	BayarConv + SRM + CNN Encoder + U-Net Decoder
Parameters	~8M trainable parameters
Training device	CPU (no GPU required)

3.15 Problems Faced

- Official ManTraNet weights unavailable — implemented architecture from scratch
- TruFor weights only available via Docker — not suitable for standalone deployment
- FaceForensics++ frames extracted to wrong folder — moved using Windows move command
- Flask 404 error — index HTML filename mismatch (indexv3.html vs index_v3.html)
- LSTM vs U-Net debate — justified U-Net choice based on spatial vs sequential nature of task

4. Data Acquisition & Ethics

4.1 Dataset Sources

Dataset	Source	Access Method	License
MICC-F220	Kaggle / University of Florence	Public download	Academic research
CASIA v2	Kaggle	Public download	Academic research
CoMoFoD	University of Zagreb	Public download form	Academic research
Columbia Uncompressed	Columbia University DVMM Lab	Public download	Academic research
FaceForensics++	TU Munich	Official request form — approved	TOS accepted, academic only

4.2 Ethics & Compliance

- All datasets are publicly available for academic research purposes
- FaceForensics++ accessed through official channel — request form submitted, TOS accepted
- No private or personal images were collected or used
- Deepfake frames from FaceForensics++ are used solely for model training, not for generating deepfakes
- Dataset images are not redistributed — only our trained model weights are shared
- All dataset licenses permit academic use without commercial restriction

4.3 Data Preparation

- All images resized to 224x224 pixels (EfficientNetB0 native input size) or 256x256 (V3)
- BGR to RGB conversion applied — OpenCV loads BGR, models expect RGB
- Pixel values normalised using ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]
- Data augmentation applied during V2 training: horizontal flip, rotation ± 10 degrees, brightness $\pm 20\%$
- Class weights computed to handle imbalance between Original and Tamper image counts
- StandardScaler applied to 1280-dimensional feature vectors before SVM training in V2
- FaceForensics++ frames extracted at FRAME_SKIP=15 to avoid near-duplicate consecutive frames

5. Algorithm Justification

5.1 Why Transfer Learning?

Training a deep CNN from scratch for image forgery detection would require millions of labelled images and weeks of GPU training time. Transfer learning uses weights pre-trained on ImageNet (1.2 million images, 1000 classes) as a starting point. The lower layers already detect edges, textures, and patterns that are universally useful. Only the classifier needs to learn the specific distinction between authentic and forged image features.

5.2 Why MobileNetV2 for V1?

MobileNetV2 was chosen for V1 because it is lightweight and fast, making it suitable for a proof-of-concept with only 220 training images. With a small dataset, a simpler model is less likely to overfit. MobileNetV2 completed feature extraction in under 2 minutes on CPU.

5.3 Why EfficientNetB0 for V2?

As the dataset grew 111x to 24,459 images, a more powerful backbone was needed. EfficientNetB0 uses compound scaling to simultaneously increase depth, width, and input resolution — achieving 5.1% better accuracy than MobileNetV2 on ImageNet with only 1.9M more parameters. For forgery detection, where subtle noise and texture inconsistencies matter, richer feature representations significantly improve classification performance.

5.4 Why SVM over Neural Network Classifier?

SVM works exceptionally well in high-dimensional feature spaces. With 1280 feature dimensions, SVM finds a maximum-margin hyperplane that generalises well to unseen images. An end-to-end neural network would require far more training data to learn both feature extraction and classification simultaneously. The SVM also provides calibrated probability scores through CalibratedClassifierCV, which are needed for the confidence percentage displayed in the web interface.

5.5 Why BayarConv + SRM for V3?

Standard CNN features describe visual content — what an image looks like. BayarConv and SRM extract noise residuals — the invisible forensic fingerprint left by the camera sensor and manipulation tools. Forged regions have a different noise fingerprint than authentic regions because they originate from a different image with different compression history and sensor characteristics. This makes V3 fundamentally more robust to forgeries that are visually convincing but forensically inconsistent.

5.6 Why U-Net Decoder instead of LSTM?

The original ManTraNet paper used LSTM to model patch-level anomaly sequences. However, our task requires pixel-level spatial localisation — producing a heatmap of the same dimensions as the input image. LSTM processes data sequentially, losing the 2D spatial structure. U-Net's skip connections from encoder to decoder preserve exact spatial coordinates, enabling precise pixel-level localisation of forged regions. U-Net is also significantly faster since it processes the full spatial map in parallel rather than sequentially.

6. GitHub Contributions

6.1 Repository Structure

Repository: <https://github.com/Manojmano954/image-forgery-detection>

Branch	Contents	Owner
main	Latest stable code (V3 merged)	Both
v1-mobilenetv2	train.py (MobileNetV2), app.py, index.html, requirements.txt	Both
v2-efficientnet	train.py (EfficientNetB0), app.py, index.html, requirements.txt	Both
v3-mantranet	train_v3.py, app_v3.py, index.html, requirements_v3.txt	Both

6.2 Contribution Split

File	Responsible	Description
train.py (V1, V2)	Manoj Kumar Konda	MobileNetV2 and EfficientNetB0 training pipelines
train_v3.py	Manoj Kumar Konda	ManTraNet-style architecture with U-Net segmentation
app.py (V1, V2)	Ranganayakulu	Flask REST API for V1 and V2 inference
app_v3.py	Ranganayakulu	Flask REST API for V3 with heatmap generation
index.html (V1, V2)	Ranganayakulu	V2 web UI with drag-and-drop and confidence bars
indexv3.html	Ranganayakulu	V3 web UI with heatmap and region detection tabs
README.md	Both	Project documentation and setup instructions
.gitignore	Manoj Kumar Konda	Excludes Dataset, saved_model, large files from GitHub

6.3 Commit Standards

All commits follow a consistent message format describing what changed and why:

- V1: MobileNetV2 + MICC-F220 dataset - copy-move detection only
- V2: EfficientNetB0 + 24459 images - Copy-Move + Splicing - 81.34% accuracy
- V3: ManTraNet-style BayarConv+SRM+UNet - Copy-Move+Splicing+Deepfake with pixel heatmap
- Fix RAM crash by loading images in batches of 16 instead of all at once
- Switch from RBF-SVM to LinearSVC for 20x faster training on large dataset

7. Reflection & Evaluation

7.1 What Went Well

- Three clear, working versions delivered on schedule with measurable accuracy improvements
- RAM crash in V2 solved creatively using batch processing — a general technique applicable to any large ML dataset
- FaceForensics++ access approved within 24 hours of submitting request form
- GitHub branch strategy effectively demonstrates evolution of the project to the assessor
- U-Net decoder successfully produces visual heatmap showing WHERE forgery occurred — key differentiator vs V1 and V2

7.2 What Could Be Improved

- GPU training would reduce V3 training time from hours to minutes
- A larger FaceForensics++ sample (200+ videos instead of 50) would improve deepfake detection accuracy
- Automated testing with a fixed test set across all three versions would provide a more rigorous comparison
- The heatmap threshold (0.6) is hardcoded — a dynamic threshold based on image statistics would improve generalisation

7.3 Key Learnings

- Transfer learning is extremely powerful — EfficientNetB0 pre-trained on ImageNet provided excellent forgery features despite never having seen forgery training data during pre-training
- Dataset quality matters more than dataset quantity — 220 high-quality MICC images with correct labels outperformed larger noisy datasets in early experiments
- Noise residual analysis (BayarConv + SRM) is fundamentally more powerful than visual feature analysis for forgery detection because it operates at the physics level of how cameras and compression algorithms work
- Publicly available pre-trained weights for specialist models are often unavailable — implementing architectures from scratch using published papers is a necessary research skill
- Documentation and regular commits throughout the project are as important as the code itself for academic assessment

7.4 Future Work

- Implement GPU-accelerated training using CUDA for V3
- Add video deepfake detection by processing frames temporally
- Deploy as a public web service using Docker and cloud hosting
- Integrate with social media APIs for automated content verification
- Train on NIST16 dataset to add object removal forgery detection as a fourth forgery type

This logbook documents the complete evolution of the Image Forgery Detection System from a 220-image baseline to a 26,000+ image multi-type detection system with pixel-level forgery localisation. All three versions are preserved on GitHub with clear commit histories demonstrating the iterative development process.